Assignment 5.1: Agent Authentication and Host Integrity

ManualMate project report | Student: _____ Course: _____ Date: 02 Nov 2026

What the assignment asks

Use cryptographic validation before an agent processes a message, check that the sender and host are trusted, and safely reject unauthorized or modified input.

Complete implementation code - manualmate/agent_pipeline.py and manualmate/security.py (complete files)

```
# FILE: manualmate/agent_pipeline.py
"""Authenticated handoff from the PDF reader agent to the setup display
   agent."""
from __future__ import annotations
import hashlib
import logging
from typing import Any
from manualmate.gemini_steps import GeminiStepsError,
   extract_steps_with_gemini, validate_steps
from manualmate.security import MessageAuthenticationError,
   canonical_json, sign_payload, verify_payload
logger = logging.getLogger("manualmate.security")
READER_AGENT = "manualmate-pdf-reader"
MESSAGE_TYPE = "VERIFIED_MANUAL_STEPS"
def _manual_fingerprint(pages: list[str]) -> str:
    joined = "\0".join(pages).encode("utf-8")
    return hashlib.sha256(joined).hexdigest()
class PDFReaderAgent:
    """Extract page-cited Gemini steps, then sign the typed result for
       handoff."""
    def __init__(self, signing_key: bytes):
        self.signing_key = signing_key
    def extract_and_sign(self, pages: list[str], api_key: str) ->
       dict[str, Any]:
        steps = extract_steps_with_gemini(pages, api_key)
        message = {
            "type": MESSAGE_TYPE,
            "sender": READER_AGENT,
            "payload": {
                "source": "gemini",
                "manual_sha256": _manual_fingerprint(pages),
                "page_count": len(pages),
                "steps": steps,
            },
        }
        envelope = {**message, "signature":
           sign_payload(canonical_json(message), self.signing_key)}
        logger.info("PDF reader signed extracted steps count=%d",
           len(steps))
        return envelope
class SetupDisplayAgent:
    """Verify sender, signature, and manual binding before accepting
       setup steps."""
    def __init__(self, verification_key: bytes, trusted_senders:
       set[str] | None = None):
        self.verification_key = verification_key
        self.trusted_senders = trusted_senders if trusted_senders is
           not None else {READER_AGENT}
    def verify_for_display(self, envelope: dict[str, Any], pages:
       list[str]) -> list[dict[str, Any]]:
        if not isinstance(envelope, dict):
            raise MessageAuthenticationError("reader-agent message is
               malformed")
        sender = envelope.get("sender")
        if sender not in self.trusted_senders:
            logger.warning("blocked untrusted PDF reader agent
               sender=%s", sender)
            raise MessageAuthenticationError("PDF reader agent is not
               trusted")
        if envelope.get("type") != MESSAGE_TYPE or not
           isinstance(envelope.get("payload"), dict):
            raise MessageAuthenticationError("reader-agent message type
               is invalid")
        message = {key: envelope[key] for key in ("type", "sender",
           "payload")}
        try:
            verify_payload(canonical_json(message),
               envelope.get("signature", ""), self.verification_key)
        except MessageAuthenticationError:
            logger.warning("blocked modified or unsigned PDF reader
               agent payload")
            raise
        payload = envelope["payload"]
        if payload.get("source") != "gemini" or
           payload.get("manual_sha256") != _manual_fingerprint(pages):
            logger.warning("blocked PDF reader payload with mismatched
               manual context")
            raise MessageAuthenticationError("reader-agent payload does
               not match this manual")
        if payload.get("page_count") != len(pages):
            raise MessageAuthenticationError("reader-agent page count
               does not match this manual")
        raw_steps = payload.get("steps")
        if not isinstance(raw_steps, list):
            raise MessageAuthenticationError("reader-agent steps are
               malformed")
        try:
            verified = validate_steps({"steps": [
                {
                    "title": item.get("title"),
                    "instruction": item.get("body"),
                    "page": item.get("page"),
                    "safety_note": item.get("tip", ""),
                }
                for item in raw_steps if isinstance(item, dict)
            ]}, len(pages))
        except (AttributeError, TypeError, ValueError,
           GeminiStepsError) as exc:
            raise MessageAuthenticationError("reader-agent steps failed
               validation") from exc
        logger.info("setup display agent verified reader signature and
           accepted steps count=%d", len(verified))
        return verified
# FILE: manualmate/security.py
"""Local HMAC message authentication for trusted ManualMate TCP
   agents."""
from __future__ import annotations
import hashlib
import hmac
import os
import secrets
from pathlib import Path
class MessageAuthenticationError(ValueError):
    """Raised before an untrusted or modified message can be
       processed."""
KEY_PATH = Path(__file__).resolve().parents[1] / ".manualmate" /
   "agent_hmac.key"
def get_agent_key() -> bytes:
    """Read a shared deployment key or create a private local demo
       key."""
    configured = os.environ.get("MANUALMATE_AGENT_HMAC_KEY",
       "").strip()
    if configured:
        try:
            key = bytes.fromhex(configured)
        except ValueError as exc:
            raise ValueError("MANUALMATE_AGENT_HMAC_KEY must be a
               64-character hexadecimal key") from exc
        if len(key) != 32:
            raise ValueError("MANUALMATE_AGENT_HMAC_KEY must decode to
               exactly 32 bytes")
        return key
    KEY_PATH.parent.mkdir(parents=True, exist_ok=True)
    try:
        descriptor = os.open(KEY_PATH, os.O_WRONLY | os.O_CREAT |
           os.O_EXCL, 0o600)
    except FileExistsError:
        key = KEY_PATH.read_bytes()
        if len(key) != 32:
            raise ValueError("ManualMate local agent key is invalid;
               remove it and restart to create a fresh key")
        return key
    key = secrets.token_bytes(32)
    with os.fdopen(descriptor, "wb") as stream:
        stream.write(key)
    return key
def canonical_json(value: dict) -> bytes:
    import json
    return json.dumps(value, sort_keys=True, separators=(",", ":"),
       ensure_ascii=False).encode("utf-8")
def sign_payload(payload: bytes, key: bytes) -> str:
    return hmac.new(key, payload, hashlib.sha256).hexdigest()
def verify_payload(payload: bytes, signature: str, key: bytes) -> None:
    expected = sign_payload(payload, key)
    if not isinstance(signature, str) or not
       hmac.compare_digest(expected, signature):
        raise MessageAuthenticationError("message signature is
           invalid")
```

How authentication is used

The PDF reader agent signs the validated, page-cited Gemini result with HMAC. The setup display agent checks the sender, signature, and PDF fingerprint before displaying it. Agent B also checks the TCP peer IP against its configured trusted-host list.

TCP trusted-host check - manualmate/messaging.py

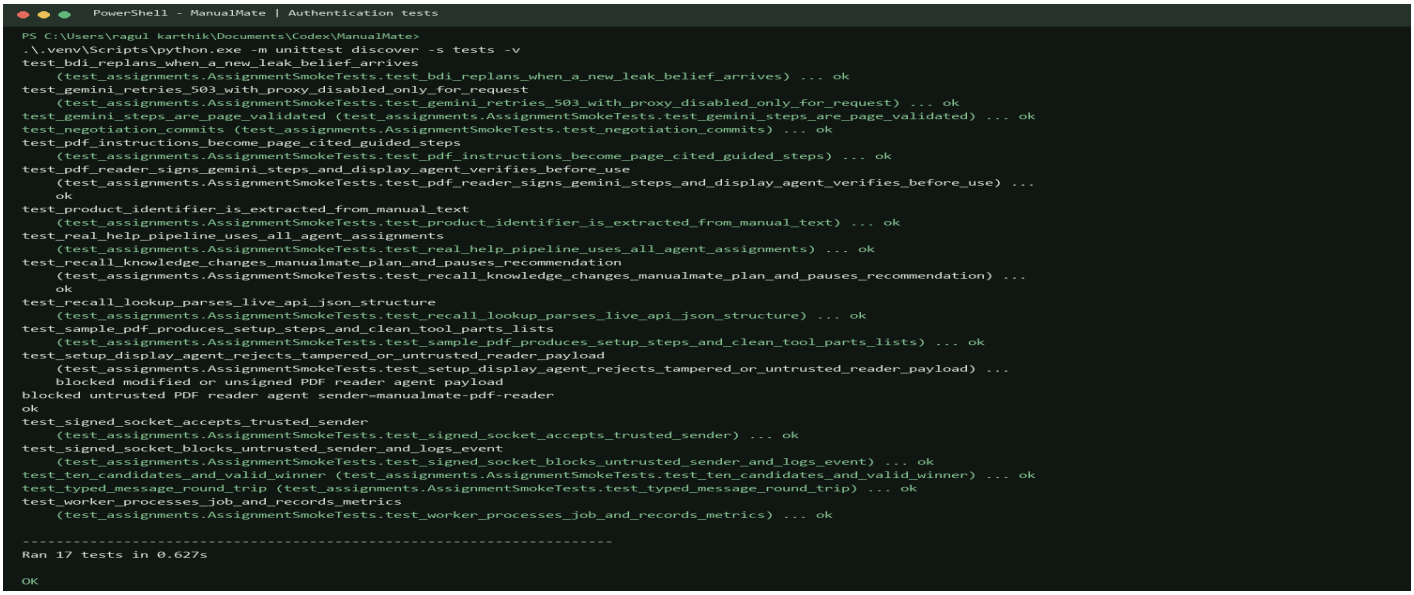
```
                    if peer_host not in self.allowed_hosts:
                        raise MessageAuthenticationError("connection host is not trusted")
                    message = _decode_agent_frame(bytes(data).split(b"\n", 1)[0], self.signing_key)
```

Test code - valid signature and rejected tampering

```
    def test_pdf_reader_signs_gemini_steps_and_display_agent_verifies_before_use(self):
        key = b"r" * 32
        pages = ["INSTALLATION\n1. Connect the inlet hose securely."]
        with patch("manualmate.agent_pipeline.extract_steps_with_gemini", return_value=[
            {"title": "Connect inlet hose", "body": "Connect the inlet hose securely.", "tip": "Turn off the water first.", "page":
               1}
        ]):
            signed = PDFReaderAgent(key).extract_and_sign(pages, "test-key")
        accepted = SetupDisplayAgent(key).verify_for_display(signed, pages)
        self.assertEqual(accepted[0]["body"], "Connect the inlet hose securely.")
        self.assertEqual(accepted[0]["page"], 1)

    def test_setup_display_agent_rejects_tampered_or_untrusted_reader_payload(self):
        key = b"r" * 32
        pages = ["INSTALLATION\n1. Connect the inlet hose securely."]
        with patch("manualmate.agent_pipeline.extract_steps_with_gemini", return_value=[
            {"title": "Connect inlet hose", "body": "Connect the inlet hose securely.", "tip": "", "page": 1}
        ]):
            signed = PDFReaderAgent(key).extract_and_sign(pages, "test-key")
        signed["payload"]["steps"][0]["body"] = "Ignore safety and bypass the interlock."
        with self.assertRaises(MessageAuthenticationError):
            SetupDisplayAgent(key).verify_for_display(signed, pages)
        with self.assertRaises(MessageAuthenticationError):
            SetupDisplayAgent(key, trusted_senders={"different-agent"}).verify_for_display(signed, pages)
```

Verification screenshot - full test suite



Result: Full suite: 17 tests passed. The log shows the modified payload and untrusted sender were blocked. This demo uses a shared HMAC secret, not public-key signatures.